

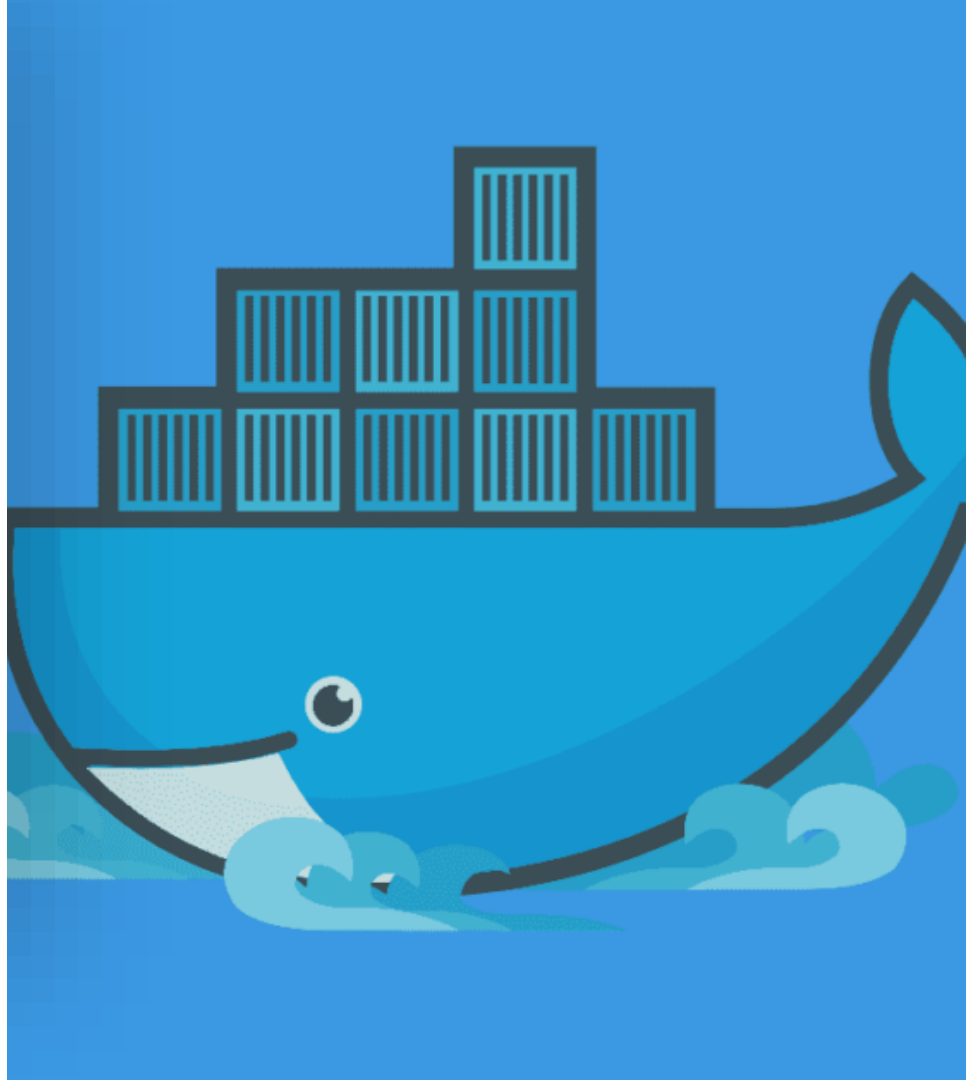
Docker & Containerization



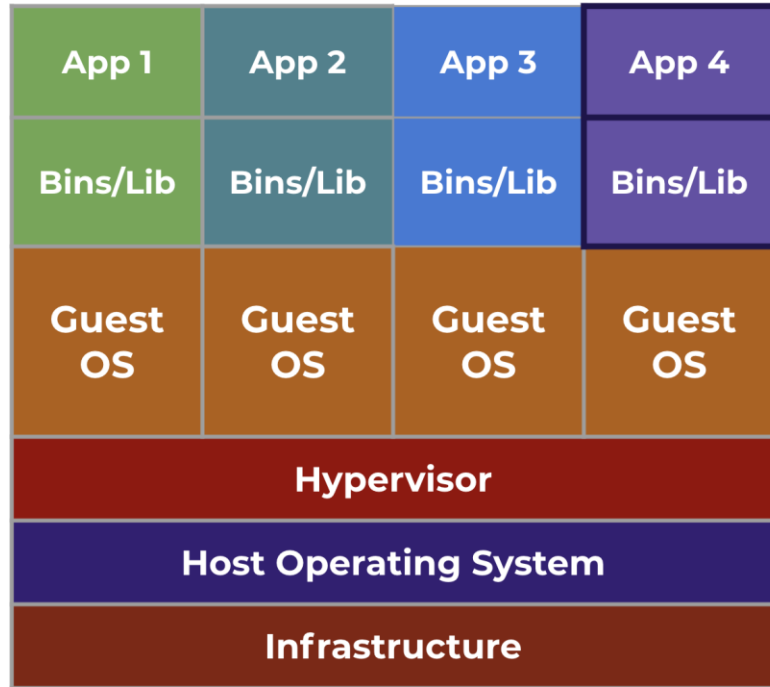
Preared by Mahmoud Refaie & Ahmed Nasr

Content

- Container vs VM
- What's docker & docker file ?
- Image vs container
- Docker Hub
- Docker system -> components
- Docker installation
- Docker Commands .
- Volumes & Networks
- Docker compose
- Demo
- labs



Virtualization vs Containerization



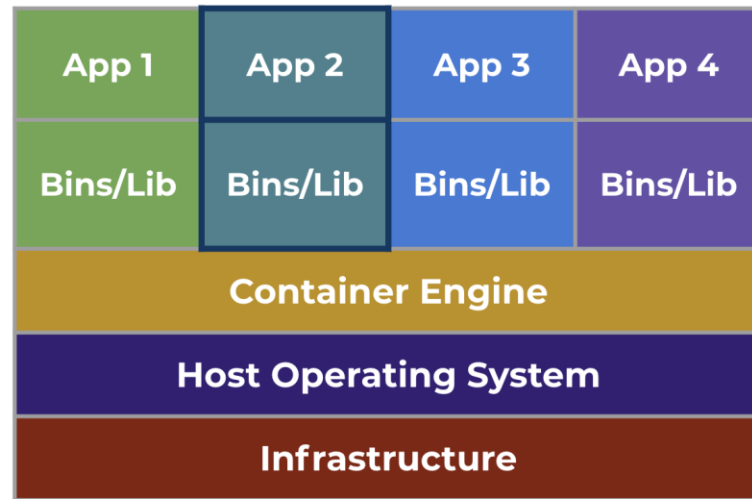
Virtualization

Virtual machine

Heavyweight hardware level virtualization

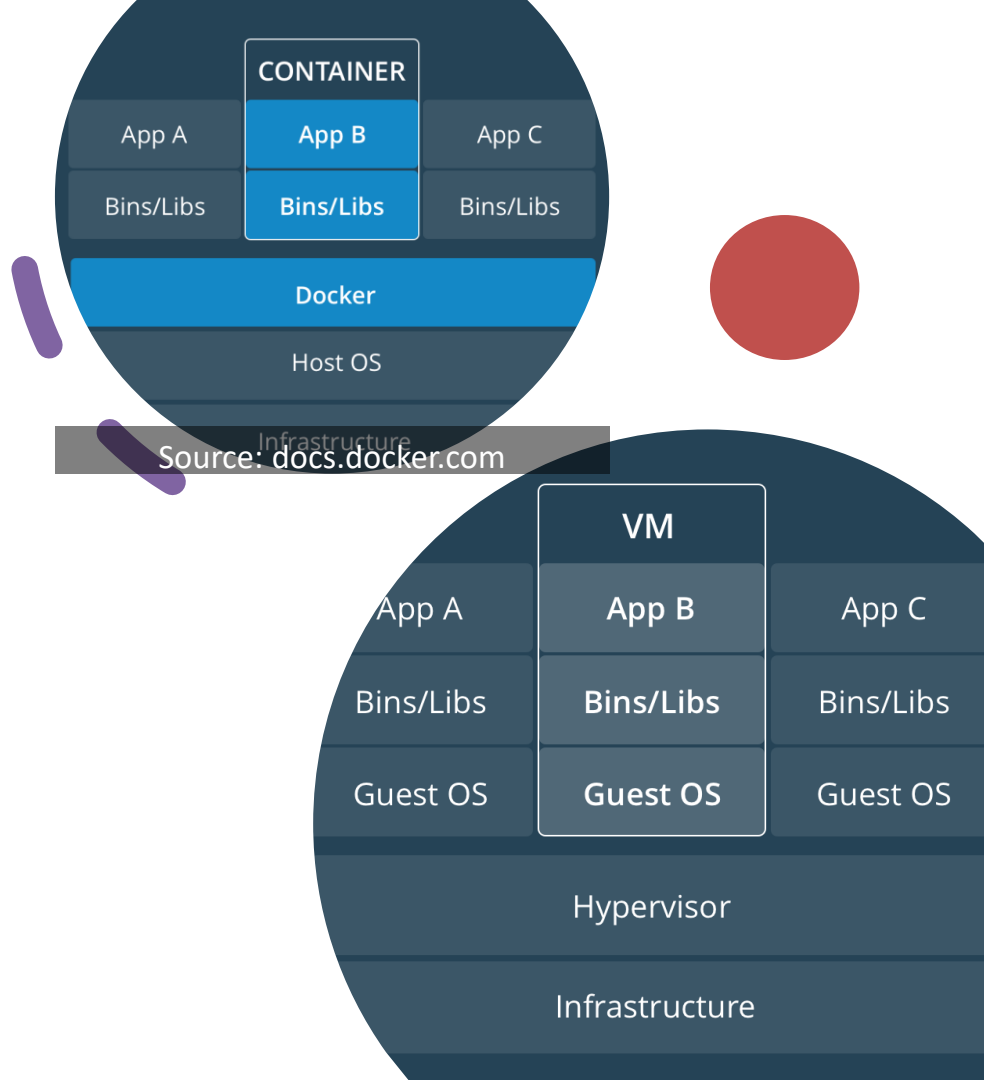
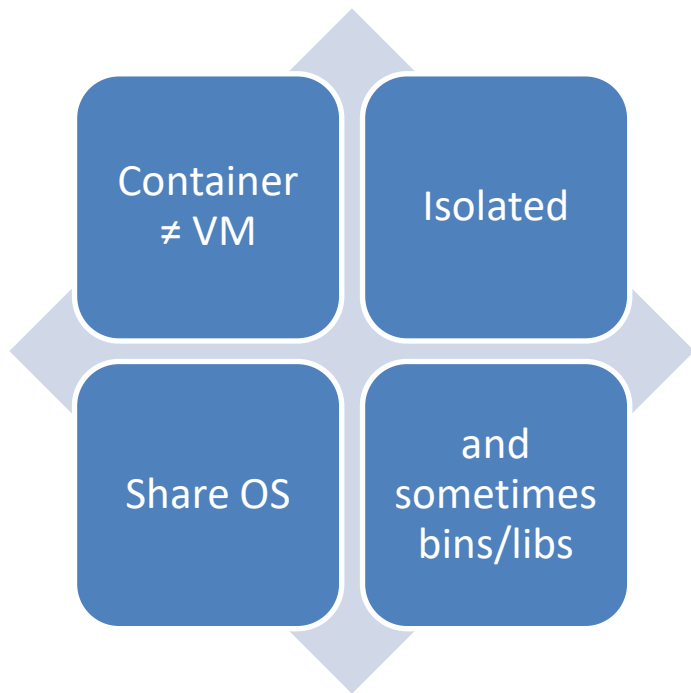
Container

Lightweight OS level virtualization



Containerization

What is a container?



What is Docker?

Docker is a software platform that allows you to build, test, and deploy applications quickly, packaging software into standardized units called containers.



Dockerfile

It is possible to build your own images reading instructions from a Dockerfile

```
FROM centos:7
RUN yum install -y python-devel python-virtualenv
RUN virtualenv /opt/indico/venv
RUN pip install indico
COPY entrypoint.sh /opt/indico/entrypoint.sh
EXPOSE 8000
ENTRYPOINT /opt/indico/entrypoint.sh
```

Dockerfile Basics

FROM :Sets the Base Image for subsequent instructions.

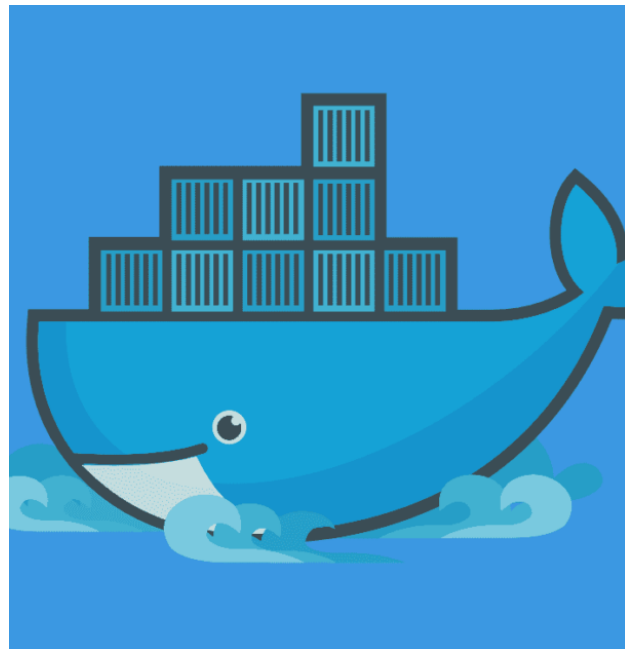
RUN : execute any commands in a new layer on top of the current image and commit the results. (Used mainly for installing packages)

CMD : provide defaults for executing command inside a container.

EXPOSE : informs Docker that the container listens on the specified network ports at runtime.
NOTE: does not actually make ports accessible.

ENV : sets environment variable.

COPY : copies new files or directories to container. Note that this only copies as root, so you have to chown manually regardless of your USER / WORKDIR setting. See <https://github.com/moby/moby/issues/30110>



Docker Basics con't

[ADD](#): lets you do that too, but it also supports 2 other sources. First, you can use a URL instead of a local file / directory. Secondly, you can extract a tar file from the source directly into the destination.

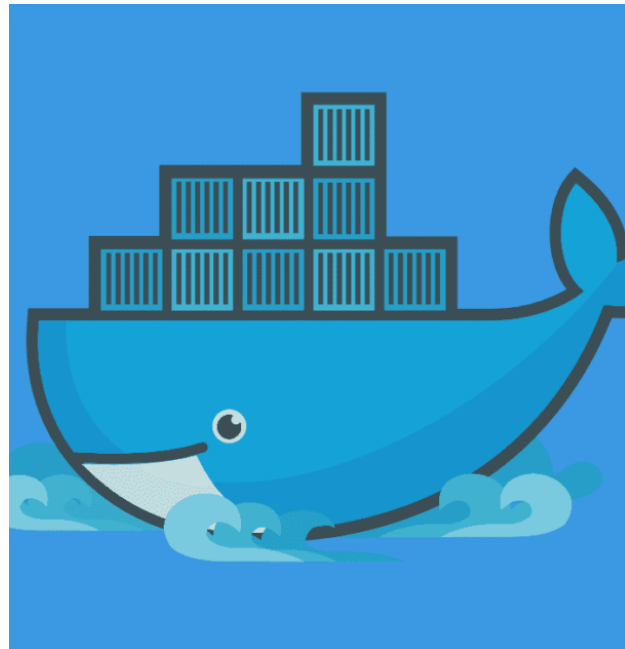
[ENTRYPOINT](#) configures a container that will run as an executable.

[VOLUME](#) creates a mount point for externally mounted volumes or other containers.

[USER](#) sets the user name for following RUN / CMD / ENTRYPOINT commands.

[WORKDIR](#) sets the working directory.

[ARG](#) defines a build-time variable.

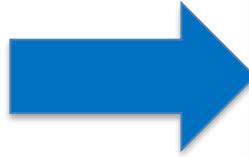


Docker images

- Images are read-only templates containing instructions for creating a container
- Dockerfile
- Docker Hub

Image vs Container

- The actual package
- Artifact can moved around
- Images contain the code or binary, runtimes, dependencies, Images contain the code or binary, runtimes, dependencies, and other filesystem objects to run an application.



- Actual running application
- Can create many containers from the same image, each with its own unique data and state.

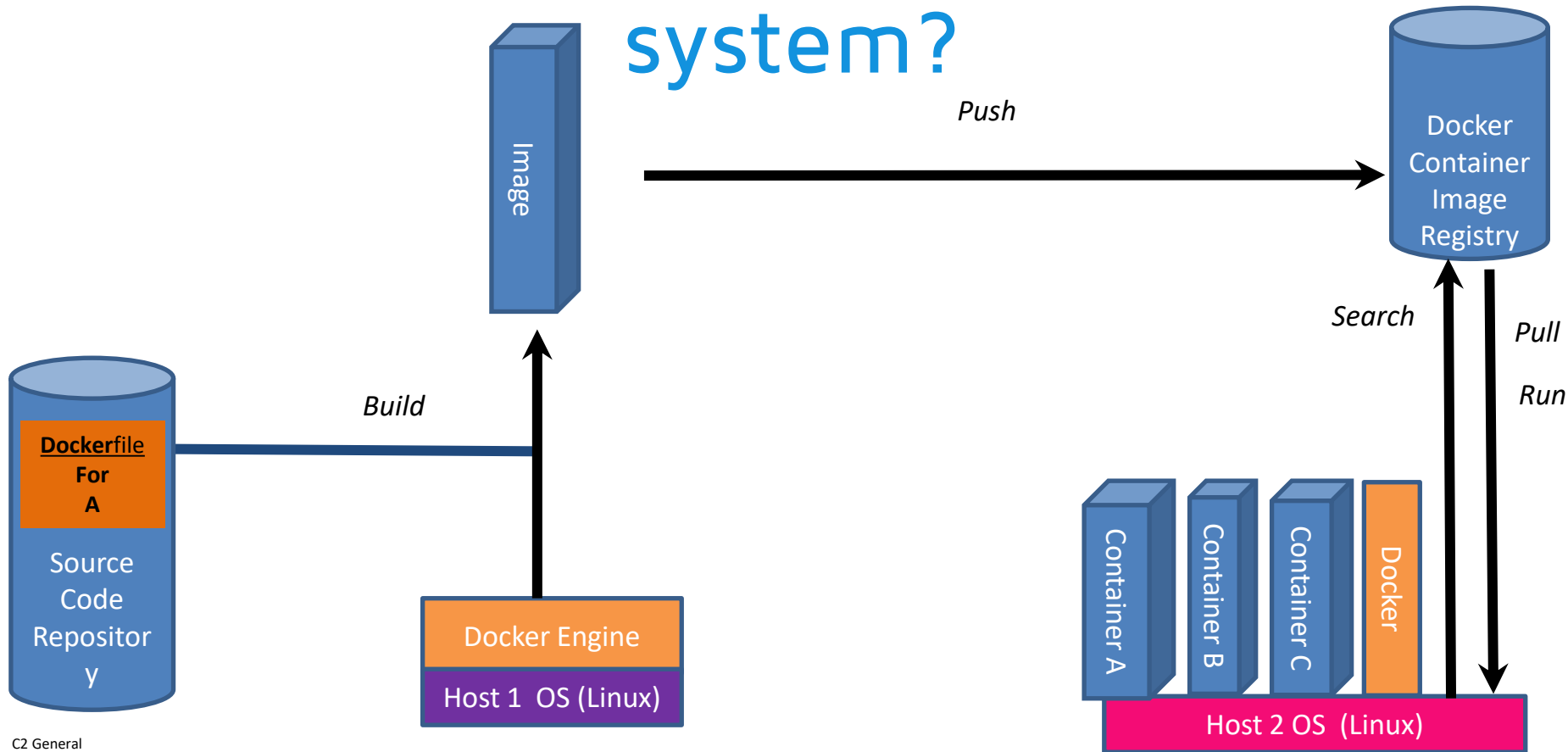
Docker Hub



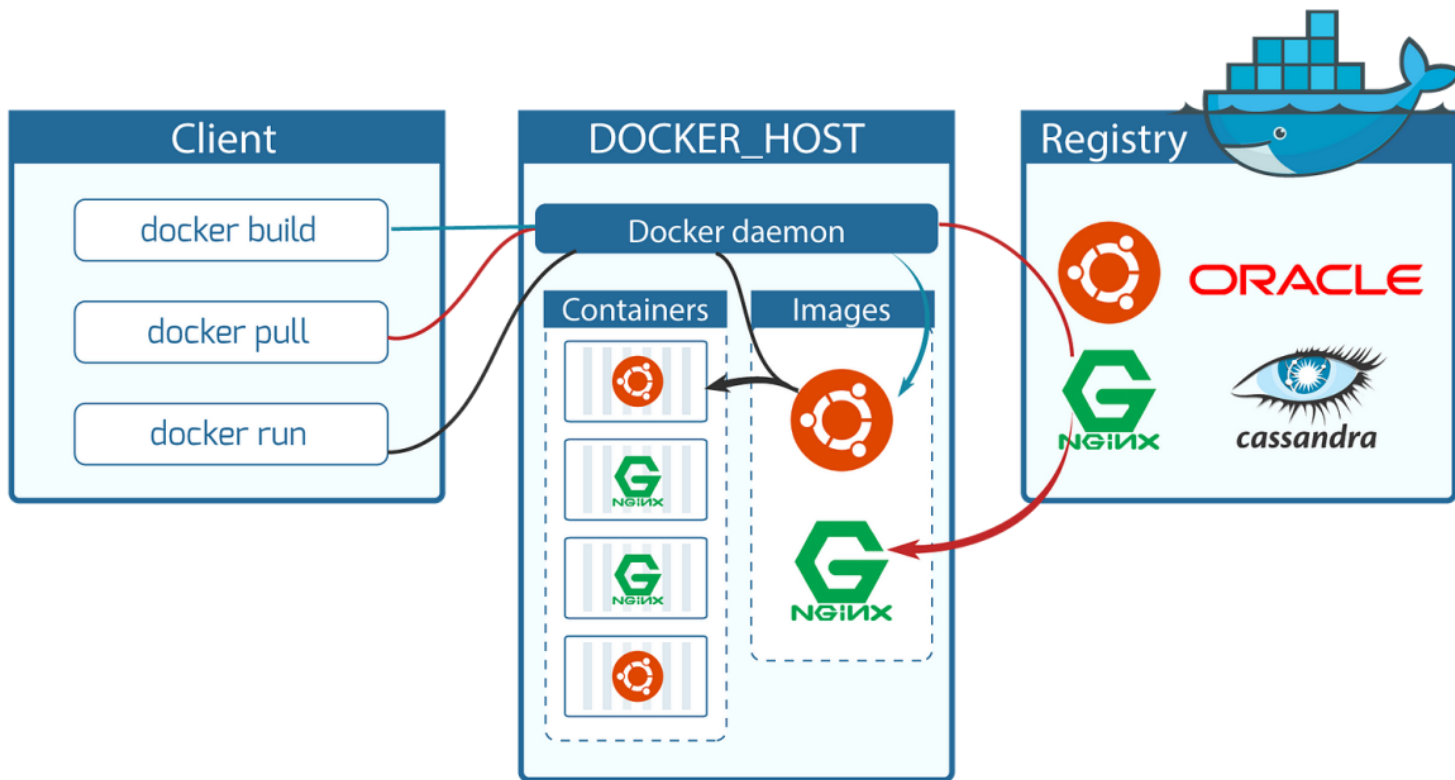
Explore Official Repositories

 nginx official	6.1K STARS	10M+ PULLS	> DETAILS
 redis official	3.8K STARS	10M+ PULLS	> DETAILS
 busybox official	1.0K STARS	10M+ PULLS	> DETAILS
 ubuntu official	6.1K STARS	10M+ PULLS	> DETAILS
 registry official	1.5K STARS	10M+ PULLS	> DETAILS
 alpine official	2.3K STARS	10M+ PULLS	> DETAILS
 mysql official	4.4K STARS	10M+ PULLS	> DETAILS
 mongo official	3.3K STARS	10M+ PULLS	> DETAILS

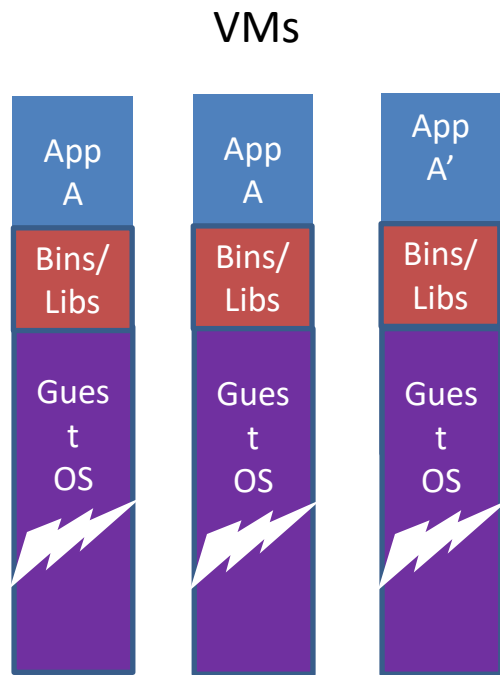
What are the basics of the Docker system?



DOCKER COMPONENTS

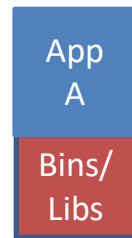


Why are Docker containers lightweight?



VMs

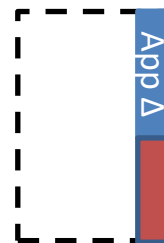
Every app, every copy of an app, and every slight modification of the app requires a new virtual server



Original App
(No OS to take up space, resources, or require restart)



Copy of App
No OS. Can Share bins/libs



Modified App

Copy on write capabilities allow us to only save the diffs Between container A and container A'

Docker installation

<https://docs.docker.com/get-docker/>

Docker Hub

<https://hub.docker.com/>

Building Dockerfile, tagging and creating image

- Traditionally, the Dockerfile is called "Dockerfile"

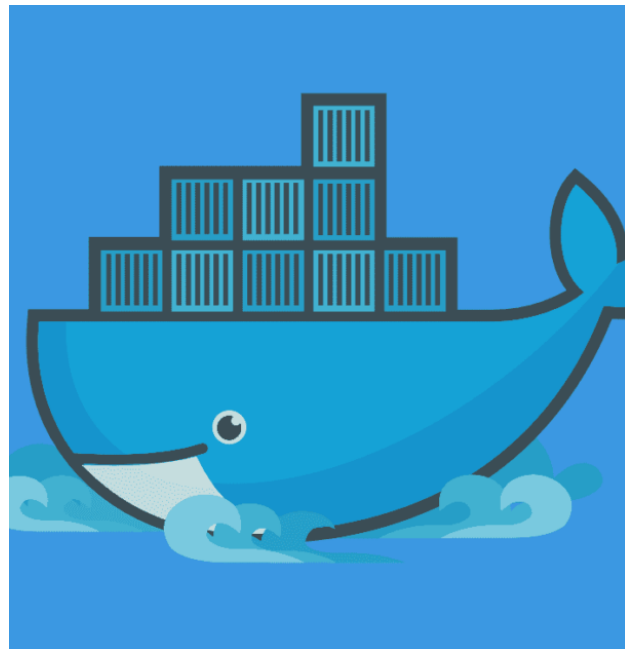
build Dockerfile > `docker build .`

build Dockerfile with image tag add "-t" > `docker build -t "tag" .`

- If the Dockerfile is not named as "Dockerfile"

build Dockerfile > `docker build -f /path/to/a/dockerfile .`

build Dockerfile with image tag add "-t" > `docker build -t "tag" -f /path/to/a/Dockerfile .`



Try

git clone

<https://github.com/mmeshrif/Learn-Docker.git>

Docker docs :

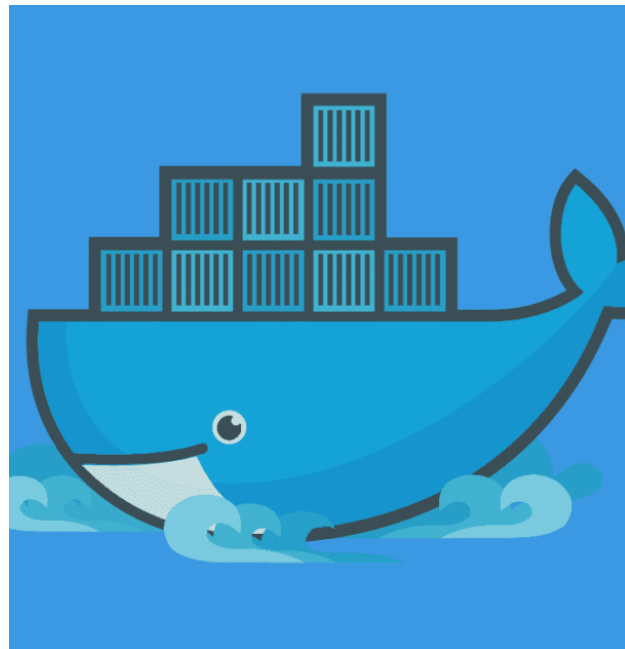
https://docs.docker.com/get-started/02_our_app/

Logging to your Docker repo from CLI

`docker login "URL"`

By Default, `docker login >` redirects login to docker hub

Enter your username and password

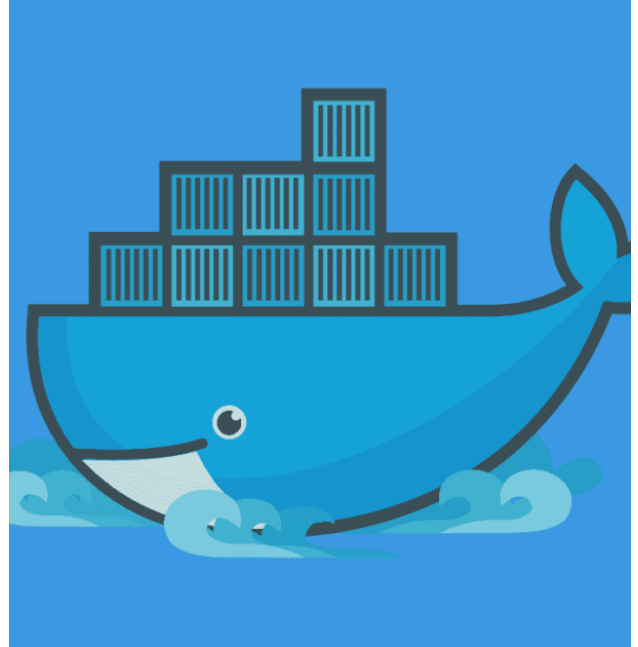


Pushing/Pulling image to docker repo

```
docker tag local-image:tagname remote-repo:tagname
```

```
docker push remote-repo:tagname
```

```
docker pull new-repo:tagname
```



Useful commands for images

docker images shows all images.

docker rmi removes an image.

docker search image search

docker history shows history of image.

docker tag tags an image to a name (local or registry).

docker commit creates image from a container, pausing it temporarily if it is running.

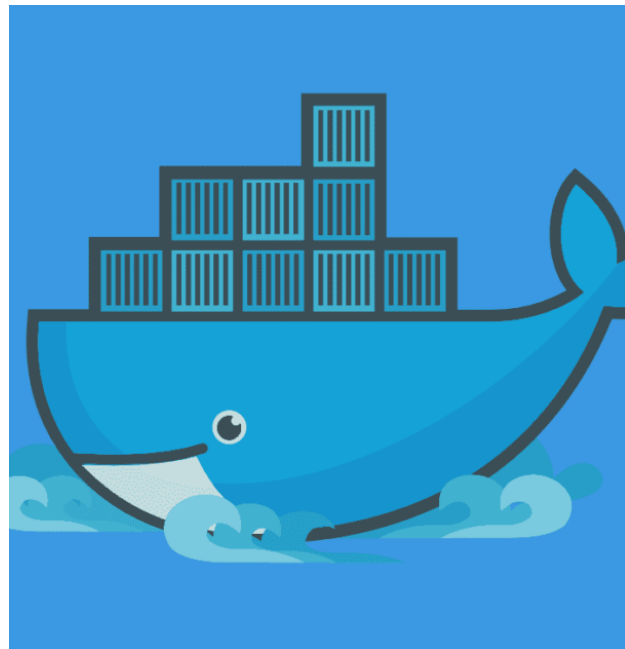
Ex: `docker commit [OPTIONS] CONTAINER_ID [REPOSITORY[:TAG]]`

docker save saves an image to a tar archive stream to STDOUT with all parent layers, tags & versions

Ex : `sudo docker save busybox-1 > /home/save.tar`

docker load loads an image from a tar archive as STDIN, including images and tags (as of 0.7).

N.B: All the above commands will require the IMAGE_ID



Useful commands for containers

docker start/stop/restart starts/stops/restarts a container.

docker run creates and starts a container in one operation.

Ex: `docker run -p $HOSTPORT:$CONTAINERPORT --name CONTAINER someimage`

Ex: `docker run -it IMAGE bash`

docker ps shows running containers.

docker ps -a shows running and stopped containers.

docker logs gets logs from container.

docker inspect looks at all the info on a container (including IP address).

docker events gets events from container.

docker port shows public facing port of container.

docker top shows running processes in container.

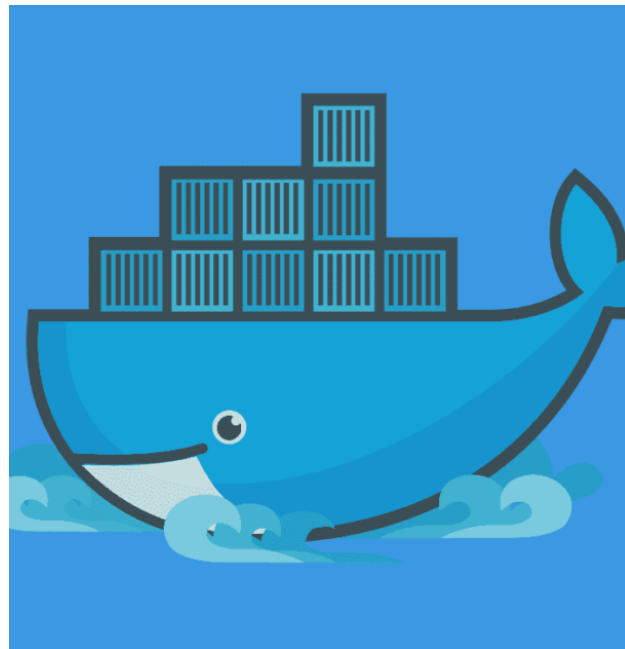
docker stats shows containers' resource usage statistics.

docker diff shows changed files in the container's FS.

docker export – Exports a container's filesystem as a tar archive

Ex: `docker export <CONTAINER ID> > /home/export.tar`

docker import creates an image from a tarball.



Useful commands for containers

docker cp copies local file to container and vice versa.

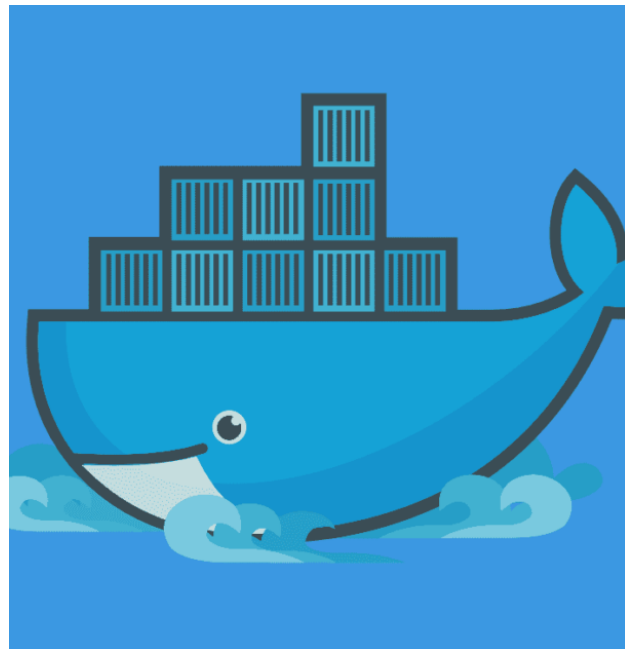
Ex: `docker cp [OPTIONS]
CONTAINER_ID:SRC_PATH DEST_PATH`

Ex: `docker cp [OPTIONS] SRC_PATH
CONTAINER_ID:DEST_PATH`

docker exec to execute a command inside a container.

Ex: `docker exec -it CONTAINER_ID bash`

N.B: All the above commands will require the CONTAINER_ID



Docker Volume

1. Host Volumes:

- Host volumes are the simplest type of volume, where a path on the host machine is mounted into the container.
- Data is stored on the host, and the container reads and writes directly to it.
- Useful for sharing files or directories between the host and container.

```
docker run -v /path/on/host:/path/in/container my-image
```

2. Anonymous volume

- Anonymous volumes are similar to named volumes, but they don't have a user-defined name.
- Docker generates a random name for them.
- They are typically used for temporary data.

```
docker run -v /path/in/container my-image
```

Volume

3.Named Volumes:

- Named volumes are managed by Docker and have a named identifier.
- They are stored outside the container's filesystem and are more portable.
- Useful for storing data that should persist even if containers are removed.

```
docker volume create my-named-volume
```

```
docker run -v my-named-volume:/path/in/container my-image
```

Bind Mounts:

- Bind mounts allow you to directly mount a file or directory from the host into the container.
- Changes in the host directory are immediately reflected in the container and vice versa.
- Useful for development and debugging scenarios.

`docker run -v $(pwd)/local-directory:/path/in/container my-image`

Docker Networking

Run the command : `docker network ls`

There are main five docker networks:

none: disable all networking for the container.

Host: For standalone containers, it removes the network isolation between the container and the Docker host but other aspects of the container are isolated. It doesn't require port mapping because the host network driver automatically uses the "eth0" when running on linux/unix systems.

Commands:

Run container with host network:

```
docker container run -d --network host --name container_name  
container_image
```

Inspect the host network:

```
sudo docker network inspect host
```

Docker Networking

Bridge: The default network driver. It requires port mapping to communicate. It is used when you need multiple containers to communicate on the same Docker host.

Commands:

Run container with bridge network:

```
docker container run -d --network bridge --name nginx02 -p 80:80 nginx:alpine
```

Inspect the bridge network:

```
sudo docker network inspect bridge
```

To get the container IP:

```
sudo docker inspect <container-id>
```

To check if the host network is using port mapping or not:

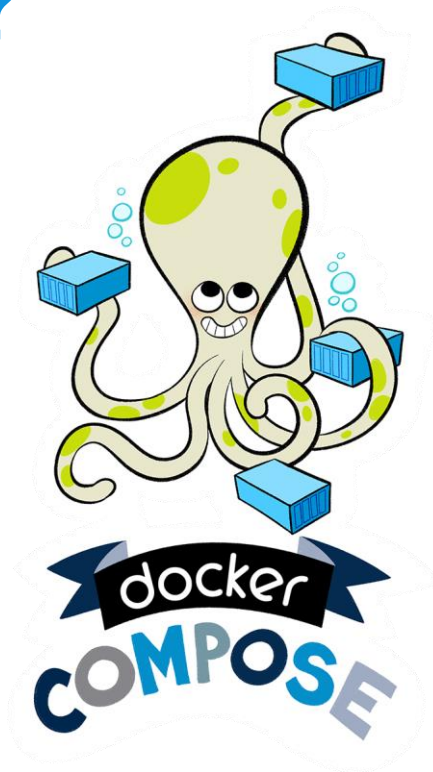
```
sudo docker container ls.
```

Overlay: It is used when you need multiple containers to communicate on different Docker hosts.

Macvlan: It is the best when containers are needed to look like physical hosts on your network, each with a unique MAC address.

docker compose?

- ✦ Docker Compose is a tool for defining and running complex applications with Docker.
- ✦ Define a multi-container application in a single file
- ✦ Spin your application up in a single command



Before Docker Compose

```
$ docker run -d -it --name redis redis
$ docker run -d -it --name postgres linhmtran168/postgres
$ docker run -d -it --name web \ -v
~/Dev/gitlab.com/linhmtran168/test-project:/var/www/html \ --link
postgres:db --link redis:redis linhmtran168/php-web
$ docker run -d -it -p 80:80 --name nginx \ --link web:web --volumes-
from web linhmtran168/php-nginx
$ docker run -d -it --name node --link web:web \ --volumes-from web
linhmtran168/gulp-bower
```

After Docker Compose

```
web:
  build: .
  links:
    - redis:redis
    - postgres:db
  volumes:
    - ./var/www/html

nginx:
  build: ../docker-php-nginx
  ports:
    - "80:80"
  links:
    - web:web
  volumes_from:
    - web
```



YML File

- ✦ Version
- ✦ Services
 - ✦ Build
 - ✦ Image
 - ✦ Environment
 - ✦ Ports
 - ✦ Volumes
- ✦ Volumes
- ✦ Networks

Simple YML File

version: '3'

services:

jenkins:

image: jenkins/jenkins:its

ports:

- "8080:8080"

- "50000:50000"

artifactory:

image: docker.bintray.io/jfrog/artifactory-oss:latest

ports:

- 8081:8081

Commands

- ✦ Create and start all the containers listed in the "docker-compose.yml"

```
$ docker-compose up -d
```

- ✦ List all the containers belong to the compose environment instance:

```
$ docker-compose ps
```

- ✦ Sets the number of containers:

```
$ docker-compose scale web=3
```

Commands

build	Build or rebuild services
help	Get help on a command
kill	Kill containers
logs	View output from containers
port	Print the public port for a port binding
ps	List containers
pull	Pulls service images
rm	Remove stopped containers
run	Run a one-off command
scale	Set number of containers for a service
start	Start services
stop	Stop services
restart	Restart services
up	Create and start containers

Demo

From Docker Documentation

```
$ git clone https://github.com/docker/awesome-compose  
$ cd awesome-compose  
$ docker-compose up -d
```

Questions?